

Session 4 — Droit et propriété intellectuelle

Objectifs de la séance

- Expliquer les mécanismes du droit d’auteur appliqués au logiciel.
- Distinguer droit d’auteur, brevets et marques.
- Identifier qui détient les droits selon le statut (salarié, stagiaire...).
- Comprendre le mécanisme du **copyleft**.
- Savoir lire la structure d’une licence logicielle.

Partie 1 — Pourquoi le droit vous concerne

Le droit vous concerne dès que vous **utilisez, contribuez, publiez ou intégrez** du code dans un cadre professionnel. Les risques concrets : une violation de licence est une **contrefaçon** (pénalement sanctionnée), elle engage la responsabilité de l’entreprise, peut rendre un produit non distribuable, et a déjà donné lieu à des **litiges coûteux** (Cisco, Orange, Vizio...).

Les licences libres reposent sur un échange : l’auteur *donne* des droits à l’utilisateur (utiliser, modifier, redistribuer), en échange de quoi celui-ci *respecte* certaines conditions. Formule canonique : « *acceptez les conditions ou refusez le don* ».

■ Ignorer une licence ne vous protège pas ; le droit n’est pas optionnel.

Partie 2 — La propriété intellectuelle

La **propriété intellectuelle (PI)** regroupe les droits exclusifs sur les créations de l’esprit. On la divise traditionnellement en deux branches :

Propriété industrielle : brevets, marques, dessins et modèles, indications géographiques.

Propriété littéraire et artistique : droit d’auteur (*copyright*), droits voisins, droits sur les bases de données.

Le logiciel est protégé principalement par le droit d’auteur, accessoirement par brevets (variable selon les juridictions) et marques (pour le nom et le logo).

Les trois piliers pour le logiciel

Protection	Objet	Durée	Formalité
Droit d’auteur	Expression, code	Vie + 70 ans	Automatique
Brevet	Invention technique	20 ans	Dépôt examiné
Marque	Nom, logo, slogan	10 ans (renouvelable)	Enregistrement

Exemples concrets : le **code source de Linux** est protégé par droit d’auteur ; le **nom « Linux »** est une marque déposée (Linux Foundation) ; un **algorithme de compression** peut, aux USA, être brevetable.

Justification et critiques

La PI repose sur un **échange de type contractuel** : l’État accorde un monopole temporaire à l’inventeur ou à l’auteur, en échange de la divulgation publique et du progrès collectif à terme. Les critiques classiques : création de monopoles temporaires, complexité juridique, coûts d’accès à la connaissance, et parfois frein à l’innovation elle-même (brevets trop larges, *patent trolls*).

Partie 3 — Le droit d’auteur appliqué au logiciel

Principes fondamentaux

Le droit d’auteur protège l’**expression** d’une idée, pas l’idée elle-même. Les œuvres doivent être **originales** (empreinte de la personnalité de l’auteur). Protection **automatique dès la création**, sans dépôt ni formalité. Pas de protection pour : les idées en tant que telles, les concepts, les méthodes, les algorithmes purs, les faits, les données brutes.

■ « *Les idées sont de libre parcours.* » — Henri Desbois

Durée : vie de l’auteur + 70 ans (UE et US pour les personnes physiques). Aux US, les œuvres de commande ou produites par des salariés ont une durée distincte : **95 ans après publication ou 120 ans après création**, au premier des deux termes échu.

Droits moraux vs patrimoniaux

Droits moraux (inaliénables en France) : droit de paternité (être identifié comme auteur), droit au respect de l’œuvre, droit de divulgation, droit de retrait (rare).

Droits patrimoniaux (cessibles) : droit de reproduction, droit de distribution, droit de modification, droit de communication au public.

Les licences libres portent sur les droits patrimoniaux. Les droits moraux subsistent : c’est pourquoi la plupart des licences imposent de **conserver les mentions d’auteur** (même en cas de fork ou de modification).

Textes de référence

- **France** : loi du 3 juillet 1985 (intégrée au Code de la propriété intellectuelle, CPI).
- **Europe** : directive 91/250/CEE, puis 2009/24/CE.
- **États-Unis** : *Computer Software Copyright Act* de 1980 — un amendement au *Copyright Act* de 1976, qui inclut explicitement les logiciels dans le champ du droit d’auteur.

Ce qui est protégé et ce qui ne l’est pas

Protégé : le code source, le code objet (binaire), la documentation, le matériel de conception.

Non protégé (ou débattu) : les APIs et les interfaces (débattu), les fonctionnalités, les langages de programmation, les formats de fichiers.

Cas emblématiques

Trois affaires structurent la jurisprudence moderne du logiciel, présentées ici dans l’ordre chronologique.

Apple vs Microsoft / HP (1988-1994) — le *look and feel* de l’interface Mac

Apple attaque **Microsoft** (Windows 2.0) et **Hewlett-Packard** (NewWave) pour violation de l’« *audiovisual copyright* » de l’interface du Macintosh. Le juge **Vaughn Walker** pose un principe structurant : seuls les **éléments d’expression originaux spécifiques** peuvent être protégés, **pas** le *look and feel* général ni les éléments **fonctionnels** (barre de menus, icônes comme métaphores de fichiers, fenêtres superposables, presse-papier...). Microsoft gagne l’essentiel des griefs en 1994 ; Apple appelle en vain.

Portée : on ne peut pas monopoliser une **convention d’interaction**. C’est exactement la même logique qui sous-tendra *Oracle vs Google* trente ans plus tard sur les APIs.

Affaire SCO vs IBM (2003-2021) — la longue agonie du FUD anti-Linux

La société **SCO Group** (ex-Caldera International, renommée en 2002) prétend avoir acquis de **Novell** les *copyrights* sur Unix. En **mars 2003**, elle attaque **IBM** pour **1 milliard de dollars** : selon SCO, IBM aurait introduit du code Unix protégé dans Linux via AIX et Dynix. Elle étend ensuite ses attaques aux **utilisateurs finaux** (DaimlerChrysler, AutoZone) pour propager la peur dans tout l'écosystème.

Arrière-plan financier : **Microsoft** achète à SCO une « licence Unix » pour environ 10 M\$ en 2003 ; **BayStar Capital** injecte 50 M\$ (prétendument à la suggestion de Microsoft). Beaucoup d'observateurs y voient un financement indirect d'une **guerre juridique contre Linux** à un moment où celui-ci menaçait les parts de marché de Windows Server.

Réaction communautaire exemplaire : le blog **Groklaw**, animé par **Pamela Jones**, documente minutieusement chaque pièce du dossier, décode la jurisprudence pour les non-juristes, et démonte pièce par pièce les prétentions de SCO. C'est l'un des tout premiers cas où une **communauté open source s'organise juridiquement**, par un journalisme d'investigation citoyen distribué.

Effondrement :

- **Août 2007** — le juge **Dale Kimball** tranche : **c'est Novell qui détient les copyrights Unix**, pas SCO. Tout l'édifice juridique s'écroule.
- **Septembre 2007** : SCO dépose le bilan (*Chapter 11*).
- **2010** : un jury confirme que Novell est bien le titulaire.
- **Août 2021** : IBM règle ce qu'il reste du contentieux avec le *trustee* de SCO pour **14,25 M\$** — clôture purement administrative.

Portée. Le cas SCO est le cas d'école du **FUD** (*Fear, Uncertainty, Doubt*) comme stratégie anticoncurrentielle : semer le doute juridique sur un concurrent pour ralentir son adoption, même quand les griefs sont vides. C'est aussi la première grande démonstration qu'une **communauté peut résister juridiquement** à une telle attaque par la transparence documentaire.

Oracle vs Google (2010-2021) — les APIs Java dans Android

Oracle rachète Sun (et donc Java) en janvier 2010, puis attaque Google en août 2010 : Android reprend environ **11 500 lignes de déclarations** (signatures, structure et organisation — la SSO) sur 37 packages Java, sans licence. La procédure est spectaculairement longue :

- 2012, juge **William Alsup** : les APIs ne sont **pas** protégeables par le droit d'auteur.
- 2014, **Federal Circuit** (appel) : les APIs *peuvent* être protégées — inversion.
- 2016, second procès : *fair use* retenu pour Google.
- 2018, Federal Circuit : *fair use* rejeté — seconde inversion.
- **Avril 2021, Cour Suprême (6-2, majorité Breyer)** : *fair use*. Quatre arguments : (1) les *declaring code* copiés sont indissociables des idées fonctionnelles qu'ils expriment — donc peu protégeables ; (2) l'usage est **transformatif** (Android = plateforme mobile nouvelle) ; (3) les 11 500 lignes ne représentent que **0,4 %** du code concerné ; (4) pas d'effet de substitution sur le marché de Java. La Cour insiste surtout sur la nécessité de **ne pas stériliser l'écosystème** en permettant la monopolisation des APIs, qui « *limiterait la créativité de futurs programmes* ». Elle **ne tranche pas** la question de fond : elle raisonne « à supposer que les APIs soient protégeables ».

Portée : les APIs *peuvent* l'être, mais leur réimplémentation propre peut être couverte par le *fair use*. Enjeu vital pour l'open source, qui pratique de longue date la réimplémentation (Wine pour Win32, Samba pour SMB/CIFS, glibc pour la libc, ReactOS...).

Note sur le *fair use* : exception du droit américain qui autorise l'usage sans autorisation dans certains cas (usage transformatif, éducatif, parodie, critique...). L'exception pour usage

transformatif **n'existe pas de manière strictement équivalente en droit français**, où les exceptions sont plus limitées (art. L.122-5 CPI).

Partie 4 — Qui détient les droits ?

Par défaut : l'auteur (personne physique) détient le droit d'auteur.

Le cas du salarié (France, art. L.113-9 CPI)

Règle spécifique au logiciel : les droits patrimoniaux sont **automatiquement cédés à l'employeur**, pour les logiciels créés dans le cadre du contrat de travail.

Cas ambigus à vérifier dans le contrat :

- Projet personnel développé sur le temps libre mais lié au métier ?
- Utilisation de ressources de l'entreprise ?
- Lien direct avec l'activité professionnelle ?

Le cas particulier des stagiaires

Point essentiel : le stagiaire n'est **pas un salarié**. L'article L.113-9 ne s'applique donc **pas**. Le stagiaire **conserve ses droits patrimoniaux** par défaut.

Conséquence : si une entreprise veut récupérer les droits sur le code écrit par un stagiaire, elle doit prévoir une **cession explicite** dans la convention de stage (ou un contrat séparé).

C'est un piège très courant, à connaître si vous faites un stage qui produit du code à valeur économique.

Partie 5 — Les brevets logiciels

Principe

Un **brevet** accorde à son titulaire un **monopole temporaire** (20 ans à compter du dépôt) sur une **invention technique**, en échange de sa **divulgateion complète** au public — tout le monde peut lire le brevet, personne ne peut le mettre en œuvre sans licence. C'est le marché de base de la PI : on troque le secret industriel contre un monopole légal et borné dans le temps.

Conditions de brevetabilité :

- **Nouveauté** — l'invention ne doit pas avoir été divulguée au public avant la date de dépôt, où que ce soit dans le monde (*absolute novelty*).
- **Activité inventive** — elle doit ne pas être évidente pour une personne du métier.
- **Application industrielle** — elle doit produire un effet technique reproductible.

Droit conféré : le titulaire peut **exclure** autrui de fabriquer, utiliser, vendre ou importer l'invention brevetée. Ce n'est pas un droit d'usage (on peut tomber sous un brevet sans le savoir en créant soi-même la même chose), c'est un **droit d'interdire**. En contrepartie, des taxes de maintien annuelles.

Pourquoi le logiciel pose un problème spécifique

Les brevets ont été conçus pour les inventions mécaniques ou chimiques — pas pour le code. Trois difficultés propres au logiciel :

1. **Frontière avec les idées abstraites** : un algorithme est un raisonnement mathématique. Or les mathématiques et les idées abstraites ne sont traditionnellement **pas brevetables**. Où tracer la ligne ?
2. **Caractère combinatoire** : un logiciel moderne assemble des milliers de techniques existantes. Si chacune peut être brevetée, le **minage** de brevets (*patent thickets*) devient impossible à auditer.

Par construction, n'importe quel programme non trivial risque d'infringer sur des brevets sans que ses auteurs ne le sachent.

3. **Cycle de vie court** : 20 ans est une éternité dans le logiciel. Un brevet qui couvre une technique de 2010 est toujours en vigueur en 2030 alors que tout l'écosystème a bougé.

Ces trois frictions expliquent pourquoi le débat « faut-il breveter le logiciel ? » est structurellement différent de celui du brevet pharmaceutique ou mécanique.

États-Unis : une évolution en dents de scie

La jurisprudence américaine a oscillé trois fois :

- **1972 — *Gottschalk v. Benson*** : la Cour Suprême refuse un brevet sur un algorithme de conversion décimal/binaire. Un algorithme pur n'est pas brevetable.
- **1981 — *Diamond v. Diehr*** : la Cour Suprême accepte un brevet sur un **procédé industriel** (vulcanisation du caoutchouc) qui utilise un algorithme. Ouvre la porte : un algorithme **appliqué à un procédé physique** peut être breveté.
- **1998 — *State Street Bank v. Signature Financial*** : la Cour d'Appel Fédérale valide un brevet sur une **méthode d'affaires** implémentée en logiciel. **Explosion** des brevets logiciels : des dizaines de milliers de dépôts par an, souvent très larges, peu examinés.
- **2010 — *Bilski v. Kappos*** : la Cour Suprême commence à freiner en rejetant un brevet sur une méthode de couverture de risque financier.
- **2014 — *Alice Corp v. CLS Bank International*** : décision majeure. La Cour Suprême établit un **test en deux étapes** : (1) la revendication porte-t-elle sur une idée abstraite ? (2) si oui, contient-elle un « *inventive concept* » qui la transforme en application concrète ? Ce test a **invalidé des milliers de brevets** logiciels et business-method dans la décennie qui a suivi.

Après *Alice*, le paysage US est moins permissif qu'il ne l'a été entre 1998 et 2014, mais les États-Unis restent **de loin** la juridiction la plus favorable aux brevets logiciels dans le monde.

Europe : exclusion théorique, pratique ambiguë

Article 52(2) de la Convention sur le Brevet Européen (CBE, 1973) : les « **programmes d'ordinateur en tant que tels** » sont **exclus** de la brevetabilité, au même titre que les méthodes mathématiques, les méthodes d'affaires et les présentations d'information.

Mais l'**Office européen des brevets (OEB)** a construit une doctrine dite du « **caractère technique** » (*technical effect*) : si un logiciel produit un **effet technique supplémentaire** au-delà de l'interaction normale avec l'ordinateur (ex. : contrôle d'une caméra, compression avec gain d'efficacité mesurable, chiffrement), alors il peut être breveté en tant qu'« invention mise en œuvre par ordinateur » (*computer-implemented invention*, CII). Résultat : **des dizaines de milliers de brevets CII** ont été accordés en Europe, malgré l'exclusion de principe. Le critère « *as such* » est devenu une ligne floue.

La directive 2005 (CII directive) aurait codifié cette pratique de l'OEB. Elle a été **rejetée** par le Parlement européen le **6 juillet 2005** par **648 voix contre 14** — vote historique après une campagne massive de la **FFII** (*Foundation for a Free Information Infrastructure*), portée notamment par des acteurs français (AFUL, April) et européens. Les brevets logiciels n'ont donc pas été explicitement codifiés en droit de l'UE ; la pratique de l'OEB perdure mais reste juridiquement contestée.

Nouveauté : la Juridiction Unifiée du Brevet (JUB / UPC) — opérationnelle depuis le **1^{er} juin 2023**. Elle centralise le contentieux sur les « brevets unitaires européens » dans la plupart des États membres. Effet probable : renforcement de l'exécution (*enforcement*) des brevets en Europe, y compris pour les CII. À surveiller.

Cas emblématiques

Cas	Années	Objet	Issue
Unisys / LZW (GIF)	1994-2004	Algorithme de compression LZW utilisé dans GIF	Unisys réclame des royalties, déclenche la création de PNG comme alternative libre
Amazon 1-Click	1999-2017	Achat en un clic	Validé aux US, rejeté en Europe (manque de caractère technique). Expiré en 2017
MPEG / H.264	1996-	Codecs vidéo	<i>Pool</i> de brevets (MPEG LA) ; royalties par appareil. À l'origine de la pression pour des codecs libres (Theora, VP9, AV1)
MP3 / Fraunhofer	1989-2017	Compression audio	Royalties versées par les encodeurs/lecteurs. Brevets expirés en 2017 – MP3 effectivement libre depuis
NTP vs BlackBerry	2001-2006	Email <i>push</i> mobile	NTP (patent troll) obtient 612,5 M\$ en <i>settlement</i> . Cas emblématique du chantage aux brevets
Apple vs Samsung	2011-2018	<i>Slide-to-unlock</i> , design	Condamnations croisées sur plusieurs continents. Apple récupère finalement 539 M\$
Alice Corp v. CLS Bank	2014	Plateforme d'échange financier	<i>Leading case</i> : invalidation des brevets abstraits mis en œuvre par ordinateur

Ne pas confondre avec *Oracle vs Google* (2021), qui concernait le **droit d'auteur** sur les APIs – pas les brevets.

Patent trolls (NPE / PAE)

Un **patent troll** – terme militant, désignation neutre : **NPE** (*Non-Practicing Entity*) ou **PAE** (*Patent Assertion Entity*) – est une entité qui **ne produit rien**, se contente d'**acheter des portefeuilles de brevets** et d'**assigner en justice** des entreprises opérantes. Modèle économique : extorsion (*settlement*) pour éviter des procès coûteux, même quand le brevet est douteux.

Acteurs emblématiques :

- **Intellectual Ventures** (Nathan Myhrvold, ex-Microsoft, fondé en 2000) – 40 000 brevets en portefeuille à son apogée ; a levé plusieurs milliards auprès de fonds de pension.
- **VirnetX vs Apple** – plusieurs centaines de millions en jugements successifs sur des brevets VPN/FaceTime.

Facteur aggravant US : pendant longtemps, les procès se concentraient dans le **Eastern District of Texas** (tribunaux jugés *plaintiff-friendly*). Le *venue* a été restreint par la Cour Suprême dans **TC Heartland v. Kraft Foods (2017)**, ce qui a fait chuter significativement l'activité des trolls.

Défenses collectives de l'open source

Face au risque, l'écosystème libre s'est organisé :

- **OIN — Open Invention Network** (2005) : *pool* de brevets cross-licencié couvrant le « *Linux system* ». Membres (>4 000) : IBM, Google, Red Hat, SUSE, Sony, Microsoft (!) depuis 2018... Tout membre s'engage à ne pas attaquer les autres sur les brevets du pool.
- **LOT Network** (*License on Transfer*, 2014) : 5 000 membres. Engagement : si l'un vend un brevet à un *troll*, les autres reçoivent automatiquement une licence — neutralise la revente de portefeuilles aux NPE.
- **Unified Patents** : organisation qui conteste les brevets trolls en procédure *inter partes review* (IPR).
- **Pledges** publiques : IBM a *pledge* 500 brevets pour l'open source (2005) ; **Tesla** a ouvert tous ses brevets (2014, déclaration de bonne foi) ; **Red Hat Patent Promise**.
- **Prior art** : bases collectives (ex. *Linux Defenders*) destinées à documenter l'antériorité et à invalider les brevets abusifs.

Épisode marquant : **en 2007, Microsoft prétend que Linux viole 235 de ses brevets**, sans jamais les lister. Classique campagne de **FUD** (cf. cas SCO ci-dessus). La réaction : fondation/renforcement de l'OIN, accord Novell-Microsoft controversé (2006), puis, in fine, adhésion de Microsoft à l'OIN en 2018.

SEP et FRAND : un cas particulier

Les **SEP** (*Standard Essential Patents*) sont les brevets indispensables pour mettre en œuvre un standard (H.264, MP3, 3G/4G/5G, Wi-Fi...). Les organismes de normalisation (**ETSI**, **IEEE**...) exigent que leurs titulaires s'engagent à les licencier à des conditions **FRAND** (*Fair, Reasonable and Non-Discriminatory*).

Enjeu pour l'open source : une licence FRAND suppose souvent une **redevance par unité** et un **accord bilatéral**, ce qui est **incompatible** avec la libre redistribution. D'où les tensions récurrentes autour des codecs (H.264, HEVC) et l'effort pour produire des alternatives *royalty-free* (**AV1** par l'*Alliance for Open Media*, Opus pour l'audio).

Débat : arguments pour et contre

Arguments en faveur des brevets logiciels :

- Protection des investissements en R&D — sans monopole temporaire, pourquoi investir ?
- Incitation à la **divulgation** (alternative au secret industriel) ;
- Valorisation des startups (un portefeuille de brevets est un actif cédable) ;
- Outil de négociation (*cross-licensing*) entre grands acteurs.

Arguments contre :

- Les **brevets sont souvent trop larges** et de piètre qualité (examen superficiel, surtout aux US pré-Alice) ;
- **Freinage** de l'innovation incrémentale (rappeler qu'un logiciel moderne combine des milliers de techniques) ;
- Coûts prohibitifs des litiges (millions de \$ par procès) ; les **PME et l'open source** sont les premiers pénalisés ;
- Les **patent trolls** détournent le système ;

- **Argument historique** : l'industrie du logiciel a innové **pendant 40 ans (1960-2000)** essentiellement sans brevets ; la période post-*State Street Bank* n'a pas produit plus d'innovation, au contraire.

Position dominante dans l'écosystème open source : hostilité de principe aux brevets logiciels, défense active par l'OIN, LOT et les *pledges*, et préférence marquée pour les licences à **clause brevets explicite** (Apache 2.0, GPL v3, MPL 2.0) — cf. session 5.

Partie 6 — Les marques

Ce qu'une marque protège

Le **nom**, le **logo**, le **slogan**, l'**identité commerciale**. La marque est une protection **distincte** du droit d'auteur : un logiciel peut être libre mais son nom reste protégé.

Exemples : « Linux » (Linux Foundation), « Firefox » et son logo (Mozilla), « Red Hat » (IBM), « Debian » (SPI).

Implications pratiques

Vous pouvez : forker le code, le modifier, redistribuer votre version.

Vous ne pouvez pas : utiliser le nom original pour votre fork, utiliser le logo, créer une confusion commerciale avec le projet d'origine.

Exemple célèbre : Debian fork Firefox et doit le renommer **Iceweasel** (2006-2016) parce que la politique de marque de Mozilla exige des builds officiels.

Partie 7 — Interopérabilité et reverse engineering

Décompilation et exception d'interopérabilité

La directive européenne 91/250/CEE interdit en principe la décompilation (qui crée une œuvre dérivée), mais prévoit une **exception pour l'interopérabilité** :

- Décompilation autorisée pour obtenir les informations d'interface,
- si elles ne sont pas disponibles autrement,
- uniquement pour les parties nécessaires à l'interopérabilité.

C'est une **spécificité européenne** importante, qui protège la concurrence et empêche les marchés captifs.

Développement *clean room*

Technique pour **créer un logiciel compatible sans contrefaçon** :

- Équipe A (*dirty room*) analyse le logiciel et documente les spécifications.
- Équipe B (*clean room*) implémente, à partir des specs seulement, sans jamais voir le code original.

Puisque les **idées** ne sont pas protégées, et que l'équipe B n'a jamais vu l'expression originale, son code est juridiquement « propre ».

Cas historiques : BIOS IBM-compatibles (Phoenix, AMI) ; **Wine** (API Windows) ; **Samba** (SMB/CIFS). Le cas de Chardet v7 vu en session 5 pose la question pour les réimplémentations par IA.

Partie 8 — Le copyleft : le « hack juridique »

Mécanisme

Le **copyleft** utilise le droit d'auteur pour **garantir** les libertés au lieu de les restreindre :

1. L'auteur détient le *copyright* sur son code.

2. Au lieu d'en faire un outil de restriction, il **accorde** les droits sous conditions.
 3. Condition principale : **toute œuvre dérivée redistribuée doit rester sous la même licence.**
 4. Les libertés deviennent **virales** et **irrévocables** ; elles se propagent avec le code.
- Le copyleft retourne le copyright **contre son objectif initial** pour garantir la liberté.

Copyleft vs domaine public

Domaine public	Copyleft
Aucune protection	Utilise la protection du copyright
N'importe qui peut s'approprier et refermer	Conditions de redistribution qui empêchent la fermeture
Pas de garantie de liberté future	Libertés garanties à perpétuité

Sans copyright, pas de copyleft possible. C'est pour cela que la FSF et les juristes du libre défendent des régimes forts de droit d'auteur, par un paradoxe apparent.

Le spectre des licences

Propriétaire → Permissif → Copyleft faible → Copyleft fort
 Fermé MIT/BSD LGPL/MPL GPL/AGPL

Critère clé : *que se passe-t-il quand vous redistribuez une version modifiée ?* Rien (permissif) → la bibliothèque reste libre (copyleft faible) → **tout** doit rester libre (copyleft fort).

Efficacité juridique

Le copyleft a été testé en justice à de multiples reprises :

- **Busybox vs nombreuses entreprises** (actions de la SFLC puis SFC).
- **GPL-violations.org** (Harald Welte, Allemagne) — nombreuses victoires.
- **Free (Freebox)** (2011, France) — condamné.
- **Orange vs Entr'Oouvert** (2011-2024, France) — Orange condamné à 800 k€ sur la base d'un usage non conforme de la bibliothèque Lasso (GPL v2).
- **SFC vs Vizio** (2021-25) — enjeux sur le droit contractuel vs droit d'auteur.

La plupart des violations se résolvent **hors tribunal** avec restitution du code source.

Partie 9 — Qu'est-ce qu'une licence ?

Définition

Une licence est un **contrat** entre l'auteur et l'utilisateur. Elle définit les droits accordés, les conditions, les limitations, les obligations en cas de redistribution. Ce qu'elle **ne garantit pas** : le prix, la qualité, le support, la maintenance, la pérennité.

Structure type

1. **Préambule** — intentions et philosophie (optionnel).
2. **Définitions** — termes employés (œuvre, code source, distribution...).
3. **Droits accordés.**
4. **Conditions** — ce que vous devez faire en retour.
5. **Limitations** — ce que vous ne pouvez pas faire.
6. **Clause de garantie** — généralement *AS IS*, aucune garantie.
7. **Clause de responsabilité** — limitation de responsabilité de l'auteur.

Pas de licence = pas de droits

- Un code sans licence **n'est pas libre**, il est **inutilisable**.

Sans licence explicite : le droit d’auteur s’applique par défaut, « tous droits réservés ». Voir le code sur GitHub ne donne aucun droit de le copier, modifier ou redistribuer.

Partie 10 — Les grandes familles

Type	Exemples	Obligation principale
Permissif	MIT, BSD, Apache 2.0	Attribution (conserver copyright)
Copyleft faible	LGPL, MPL	Partager les modifications de la lib
Copyleft fort	GPL, AGPL	Tout le projet dérivé sous même licence

Chaque famille sera étudiée en détail en **session 5**.

Ce qu’il faut retenir

1. Le logiciel est protégé par le **droit d’auteur** — pas besoin de dépôt.
2. Les **brevets logiciels** existent surtout aux US ; exclus « en tant que tels » en Europe.
3. Les **marques** protègent les noms et logos, même pour des logiciels libres.
4. Le **copyleft** utilise le droit d’auteur pour **garantir** les libertés.
5. En entreprise, **les salariés cèdent automatiquement, les stagiaires non** (à négocier).
6. Une licence est un **contrat** : pas de licence = pas de droits.

Tableau récapitulatif des protections

Protection	Objet	Durée	Formalité	Exemples
Droit d’auteur	Expression, code	Vie + 70 ans	Automatique	Code source, documentation
Brevet	Invention technique	20 ans	Dépôt examiné	Algorithme (US)
Marque	Signe distinctif	10 ans renouvel.	Enregistrement	« Linux », logo
Secret	Information	Illimité	Aucune	Code propriétaire

Pour aller plus loin

- Code de la propriété intellectuelle (partie logiciels) : https://www.legifrance.gouv.fr/codes/texte_lc/LEGITEXT000006069414/
- Droit des Logiciels (F. Pellegrini et S. Canevet, 2013): <https://www.puf.com/droit-des-logiciels>
- Livret bleu “Fondamentaux juridique”: https://cnll.fr/media/LivretBleu_Juridique-2eEdition_GT-LogicielLibre_Systematic_Nov2016_web.pdf